CS 690: Software Development Project

# Smart Campus Parking

Milestone 3: Minimum Viable Product

Prepared by
**Samarth Thakor and Nirmal Naik**

Class demonstration: October 22, 2026
Submission due: October 23, 2026

**Project repository**
github.com/SamarthThakor/smart-campus-parking

**Milestone 3 implementation**
Pull request #1 - feature/milestone-3-mvp

The MVP connects student registration, permit selection and class scheduling to persistent records and a parking recommendation. An administrator can close a lot, and the next student request uses that saved change.

Prepared and tested October 20, 2026. The code and local demonstration evidence are complete. The scheduled classroom demonstration and actual peer/instructor feedback are pending; Section 4 must be updated after class before final submission.

Naming update: this milestone uses Smart Campus Parking. Earlier submissions used Smart Campus Parking & Schedule Assistant. The shorter title does not remove the class-schedule feature.

# 1. Vertical Slice Implementation

## 1.1 Complete high-priority workflow

The main slice implements **US-03: add, edit and remove weekly classes**. It is supported by US-01 (account), US-02 (permit), US-04 (next class) and US-05 (parking recommendation). The form sends a real HTTP request; Flask validates it and commits a class record. The schedule and recommendation are then read back from the database.

| Step | User action and system response |
|---|---|
| 1 | Create a student account. POST /api/auth/register stores a unique email and password hash; the browser then signs in. |
| 2 | Select Student and save the permit. PATCH /api/me/permit persists permit_id on the signed-in user. |
| 3 | Enter Database Systems, tomorrow, 10:00-11:00, Library Building. POST /api/classes validates the values and inserts the class. |
| 4 | The page reloads the schedule and recommendation through GET requests. Lot B is returned with 18 spaces and a five-minute walk. |
| 5 | Restart Flask and sign in again. The saved account, permit and class are read from the same database file. |
| 6 | An administrator closes Lot B. After the student clicks Refresh parking, Lot A becomes the recommendation. |

## 1.2 System interaction diagram

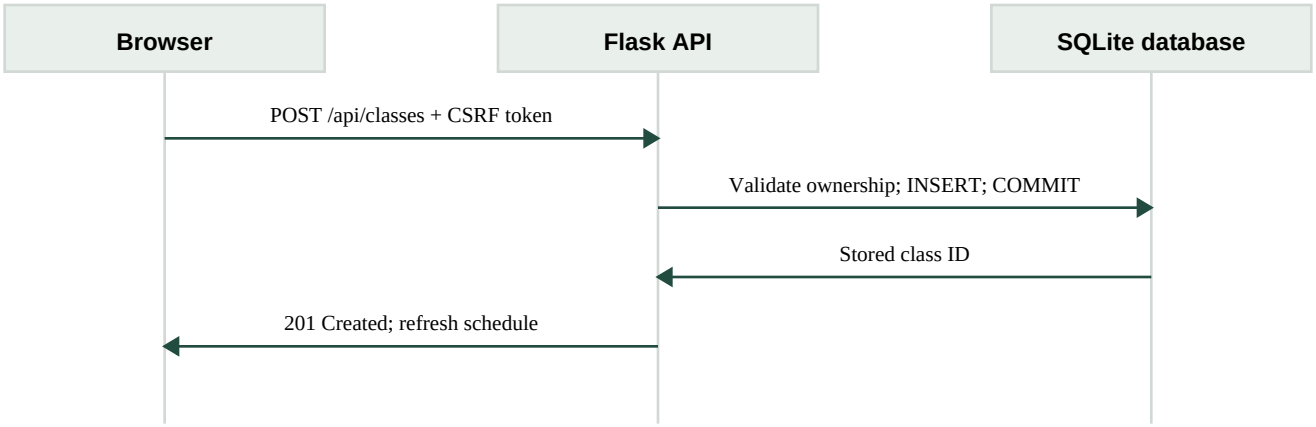



Figure 1. The write succeeds only after the transaction commits. Later GET requests and a restarted server use the persistent record.

The administrator slice follows the same path: authorized PATCH request, parking_lot update plus lot_update audit entry in one transaction, then a fresh student GET. Student sessions receive 403 for administrator routes.

# 1. Vertical Slice Evidence

## 1.3 Saved schedule and recommendation

The screenshots below come from the running browser against a live local Flask server. The new student account was created through the registration form; this is not a static page or a hard-coded recommendation.

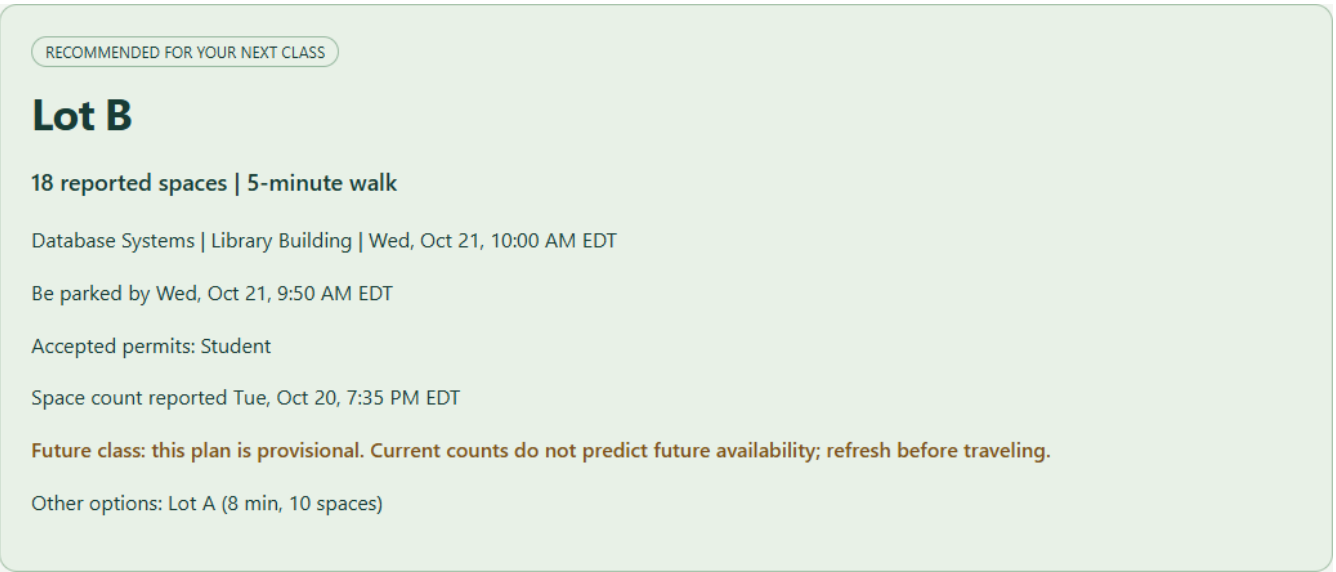



Figure 2. After saving a class and Student permit, the service recommends Lot B. The next-day plan is labeled provisional because current counts do not predict future availability.




Figure 3. The created class is returned from the database and appears with Edit and Delete controls.

## 1.4 Persistent row check

```
SELECT c.title, u.email, u.permit_id
FROM class_session c JOIN user u ON u.user_id = c.user_id
WHERE u.email = 'mvp@example.test';

Database Systems | mvp@example.test | 1
```

The automated browser check also stops the server process, starts a new one using the same database file, signs in again and verifies that Database Systems and the Lot B recommendation are still displayed. Editing and deletion are exercised afterward on this isolated test account.

Evidence: docs/ms3-evidence/result.json and startup.log. Full-page restart evidence is included in the source package. The browser test database is separate from the normal demo database.

# 1. Administrator Update Evidence

## 1.5 Change a lot and refresh the student view

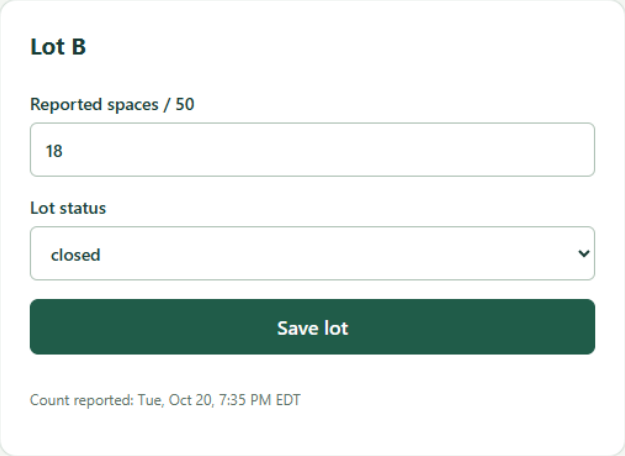



Figure 4. Lot B is stored as closed after the administrator saves.

The administrator selects **closed** for Lot B and clicks **Save lot**.

Flask checks the session role, validates the update, and commits the lot status and audit row together.

The space-count timestamp is not refreshed by a status-only change. This keeps the reported count age accurate.

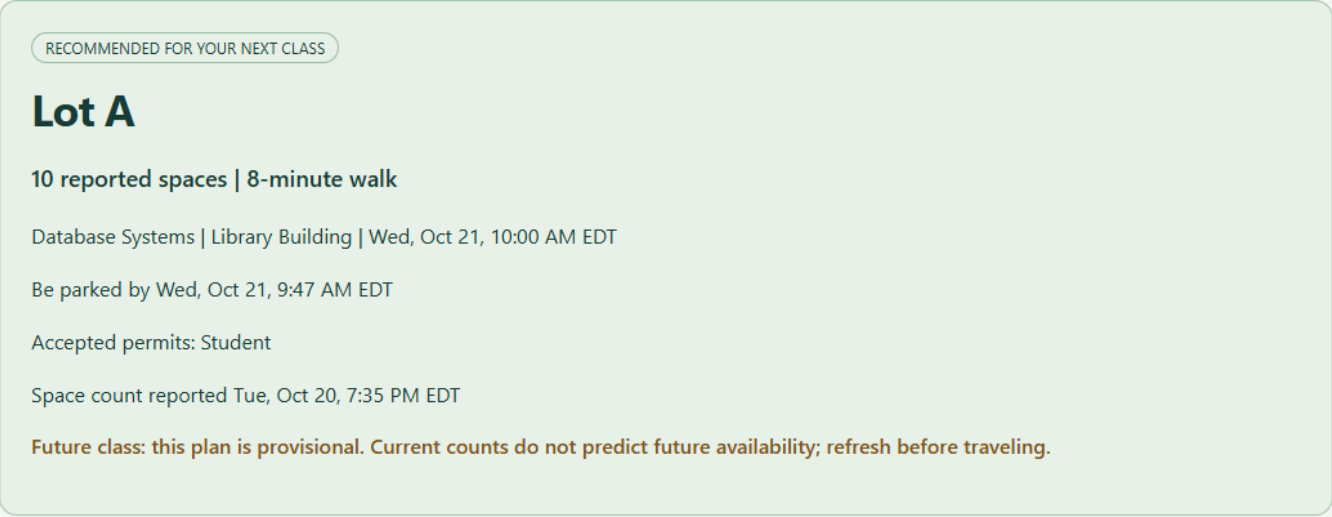



Figure 5. The student refreshes the page data. Lot A replaces Lot B because closed lots are filtered out before ranking.

| Check | Observed result |
|---|---|
| Persistent lot state | parking_lot.lot_id = 2 has status = closed. |
| Audit record | One row records lot_id = 2, actor_id = 2 and the closed status change. |
| Student result | Lot A; 10 reported spaces; eight-minute walk. |
| Invalid change | An end time before the class start is rejected; the saved class is unchanged. |

# 2. Data Storage and Management

## 2.1 Live persistent database

The MVP uses a file-backed SQLite database through SQLAlchemy. Its normal path is instance/parking.db. Records survive application restarts. Sample locations are fictional seed data, but account creation, class edits and lot changes are real database transactions. MySQL/RDS and AWS remain later deployment targets; this report does not claim they are deployed or tested.

| Table | Purpose and relationships |
|---|---|
| user / permit_type | Unique email and hashed password; role is student or admin. user.permit_id references a valid permit and may initially be null. |
| class_session / building | Each class references its owner and building. A user may have many weekly class entries. Deleting a user cascades to that user's classes. |
| parking_lot / lot_permit | Stores capacity, spaces and status. Composite keys prevent duplicate lot-permit pairs. Both references must exist. |
| walk_estimate | One positive walking-time estimate per lot/building pair, enforced by a composite primary key and foreign keys. |
| lot_update | Records administrator changes with references to the lot and acting user, a timestamp and the submitted changes. |
| alembic_version | Tracks the installed migration revision so completed migrations are not repeated. |

## 2.2 Integrity and transaction rules

SQLite foreign keys are enabled on every application connection. Database checks restrict roles, weekdays, lot status, positive walking minutes and space counts between zero and capacity. Class end times must follow start times. Required columns are non-null and emails are unique.

Flask validates JSON and referenced IDs before writes. Class edits are restricted to the signed-in owner, and lot updates require an administrator session and CSRF token. Invalid operations roll back. Passwords are hashed; responses do not expose password hashes.

## 2.3 Preservation and operating limits

Back up the local database while the server is stopped. A restored copy can be selected with DATABASE_URL and upgraded at startup. Existing unversioned MS2 files are preserved and rejected with a clear message; use a separate new database file for this MVP. Automatic import of old unversioned data is not implemented.

The local MVP uses one application process. Before production, migrations should run once as a deployment step, followed by multiple workers if needed. MySQL validation, hosted backups, HTTPS and production load testing remain future work.

# 2. Database Migration Evidence

## 2.4 Versioned changes run during startup

The application factory creates the database engine, enables foreign keys, and calls upgrade_database(engine) before registering routes. Alembic reads the stored revision and applies only pending migrations. A migration failure prevents normal startup.

```
config = Config()
config.set_main_option("script_location", migrations_path)
with engine.begin() as connection:
    config.attributes["connection"] = connection
    command.upgrade(config, "head")
```

Excerpt from app/migrations.py; migrations_path is the resolved migrations directory.

| Revision | Executed change |
| --- | --- |
| 0001_initial | Creates seven core tables from a frozen initial schema, including keys and constraints. |
| 0002_lot_audit | Adds lot_update with foreign keys to parking_lot and user, preserving existing records. |

## 2.5 Captured startup log

```
Applying 0001_initial: create seven core tables
Applying 0002_lot_audit: create lot_update audit table
Database migration complete: 0002_lot_audit
Demo data loaded. Local demo password: DemoParking123!

# First server start and subsequent restart:
Database migration complete: 0002_lot_audit
```

These lines are recorded in docs/ms3-evidence/startup.log. Initial setup reaches the latest revision; repeated starts report the same revision without rerunning the schema changes.

## 2.6 Upgrade and persistence checks

A dedicated integration test creates revision 0001, seeds records and changes Lot B to seven spaces. Starting the current application applies revision 0002. The test confirms that the seven-space count and student email remain and the audit table now exists. A separate test confirms that a saved class and changed permit survive a new application instance.

## 2.7 Reproduce the local MVP

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements-dev.txt
Copy-Item .env.example .env
.venv\Scripts\python -m flask --app app:create_app seed-demo
.venv\Scripts\python -m flask --app app:create_app run
```

Run seed-demo once in a fresh database, then open http://127.0.0.1:5000. The README includes platform alternatives, accounts, configuration and the demonstration sequence.

# 3. Initial Testing and Quality Checks

## 3.1 Unit and integration report

Local execution on October 20, 2026: **42 tests passed, zero failures and zero errors**. Statement coverage is **95.17%** (315 of 331 statements). The measured run took 52.92 seconds. The configured minimum is 85%.

| Area | Checks and result |
|---|---|
| Accounts and permissions | Registration, password hashing, login/logout, CSRF, ownership and administrator access passed. |
| Parking and schedule logic | Permit filtering, closed/full lots, ranking, week boundaries, no-match results and class CRUD passed. |
| Validation and integrity | Invalid counts, class values, permits and foreign-key writes are rejected; stored values remain unchanged. |
| Migrations and restart | Populated 0001-to-0002 upgrade, repeated startup, saved permit/class persistence and unversioned-file protection passed. |
| Browser integration | 14 workflow checks passed through a real browser and HTTP server, including restart, admin update, edit/delete and mobile-width layout. |

## 3.2 Coverage summary

| Component | Statement coverage |
|---|---|
| app/routes.py | 96.08% |
| app/services.py | 90.74% |
| app/migrations.py | 100.00% |
| app/models.py | 100.00% |

## 3.3 Pull-request continuous integration

The Build and test workflow runs on push and pull_request. It installs Python 3.12 dependencies, compiles the code, runs pytest with coverage, installs Chromium, and runs the browser workflow. Test reports, screenshots and logs are uploaded as a workflow artifact, including on failure.

**Observed hosted result:** pull-request run #3 passed for commit c5e856b. Trigger: pull_request. Open the GitHub Actions run.

Reproduce with python scripts/check.py and python scripts/browser_demo.py. Reports are saved under artifacts/; captured submission evidence is under docs/ms3-evidence/. Statement coverage does not establish full branch coverage, production capacity or complete accessibility compliance.

# 4. Midterm Demonstration and Peer Feedback

## 4.1 Live demonstration plan - October 22, 2026

Samarth will introduce the parking problem and demonstrate student registration, permit selection, and class entry. Nirmal will explain the persistent database, startup migrations and test results. The team will then show an administrator closing Lot B and the student receiving Lot A after refreshing.

The five-minute sequence is: introduce the problem; save a new schedule; show the recommendation; restart and confirm persistence; close the lot and refresh; show an invalid class-time error and the passing CI report. The team should restore Lot B to open after the demo.

## 4.2 Feedback record - pending classroom session

**As of October 20, the scheduled classroom demonstration has not occurred.** No peer or instructor feedback has been received for that session. This section must be completed with actual comments after the October 22 demonstration and before submitting the PDF on October 23.

| Source | Actual feedback | Follow-up |
|---|---|---|
| Instructor | Pending October 22 demonstration. | Record the comment, decision, owner and acceptance check. |
| Peers | Pending October 22 demonstration. | Record usability concerns and requested improvements. |

## 4.3 Proposed priorities for the rest of the semester

The priorities below are a preliminary team plan, not feedback already received. After class, the team will adjust the order based on observed problems and the instructor's comments.

| Priority | Proposed action | Lead |
|---|---|---|
| 1 | Fix any failure in the core workflow and repeat the regression checks before adding features. | Both |
| 2 | Improve forms, wording and navigation based on real demonstration feedback. | Samarth |
| 3 | Verify MySQL migrations and an AWS deployment path, including secrets and backup handling. | Nirmal |
| 4 | Consider the optional map after core reliability and usability work. | Both |

For each feedback item, record whether it is required for reliability, useful for usability or optional. Assign an owner and a testable completion condition. The editable checklist and feedback table are in docs/DEMO_AND_FEEDBACK.md.

Submission source: the provided CS 690 Milestone 3: Minimum Viable Product assignment (MS3.pdf). All screenshots and test results in this report come from this implementation; sample campus data is fictional.